

THE ML / DL HANDBOOK · COMPANION SERIES, VOLUME VIII

The Transformer

The Block, the Stack, and the Two Halves

Residuals & LayerNorm

The feed-forward network

The encoder block

Cross-attention

BERT vs GPT

Chapter 21 · From one attention layer to a working architecture

01

What attention alone cannot do

Three things missing from Volume VII, and what happens without each

02

The wrapper: residual and LayerNorm

Why every sub-layer is written the same two lines

03

The feed-forward network

The other two-thirds of the parameters, and the other half of the job

04

The encoder block, and the stack

Shapes, parameter counts, and what depth actually buys

05

The decoder and cross-attention

Three sub-layers, three kinds of attention, one bridge between the stacks

06

Training, inference, and the three families

Teacher forcing, the KV cache, and why almost everything is decoder-only now

One attention layer is not a model

Self-attention solved the mixing problem completely. Stacked on its own, it still fails — and it fails in three specific, separable ways.

1 • It has no depth-wise capacity

Attention is a weighted *average* of value vectors.

Averaging is a linear operation — stack two of them and you get another weighted average. There is almost no per-position non-linearity anywhere in the layer.

2 • It degenerates with depth

Stack pure attention and every token vector converges towards the same vector — doubly exponentially fast in the number of layers. A deep stack of attention alone has nothing left to distinguish its positions with.

3 • It is unstable to train deep

Six, twelve, ninety-six blocks of anything is a deep network. Without a protected gradient path and a normalisation step, the usual depth problems from Volume V arrive on schedule.

WHAT THIS CHAPTER ADDS

Three things, all of them small: a **residual connection** around every sub-layer, a **LayerNorm** after it, and a two-layer **feed-forward network** applied at each position. Together with the attention layer of Volume VII they make one block — and the block is the whole architecture, repeated.

Everything is written as a wrapped sub-layer

The Transformer has exactly two kinds of sub-layer, and both are wrapped identically. Learn the wrapper and there is nothing else to learn about the block.

1

Keep a copy of x

Before the sub-layer touches it, the input is set aside — it will be added back unchanged.



2

Run the sub-layer

Either multi-head attention or the feed-forward network. Its output has the same shape as its input, $n \times d$.



3

Add, then normalise

$x + \text{Sublayer}(x)$, passed through a LayerNorm. Out comes an $n \times d$ matrix, ready for the next wrapper.

THE WHOLE BLOCK, IN FOUR LINES OF PSEUDO-CODE

```
x = LayerNorm( x + MultiHeadAttention(x, x, x) )
```

```
x = LayerNorm( x + FeedForward(x) )
```

That is an encoder block. The decoder block inserts one more wrapped sub-layer between the two.

WHY THE SHAPE MAY NEVER CHANGE

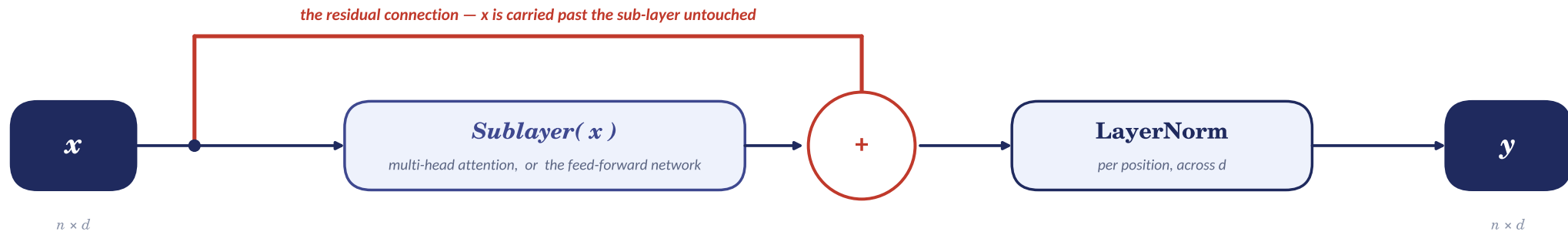
The residual connection has to add x to $\text{Sublayer}(x)$, so the two must have the same shape. That single constraint fixes d — 512 in the original paper, 768 in BERT-base, 4096 in a 7B model — for the entire depth of the network, and it is what makes the blocks interchangeable enough to stack ninety-six of them.

FIGURE 21.1

The wrapper

Two lines of arithmetic, drawn.

Every sub-layer in a Transformer is wrapped in the same two lines.



$$y = \text{LayerNorm}(x + \text{Sublayer}(x))$$

WHY THE SHAPE NEVER CHANGES

Every sub-layer maps $n \times d$ to $n \times d$. That is what makes the block stackable — and it is forced by the residual connection, which has to add x to $Sublayer(x)$. It is also why d is fixed at 512 (or 768, or 4096) throughout the entire model.

Figure 21.1: The red path is the residual connection. Note that it bypasses the sub-layer entirely — whatever the sub-layer does or fails to do, x arrives at the addition intact.

One wire, two entirely different jobs

Residuals arrived in 2015 to make very deep CNNs trainable. In a Transformer they do that too — but they are also load-bearing for the representation itself.

JOB ONE — A ROAD HOME FOR THE GRADIENT

Differentiate $y = x + F(x)$: $\partial y / \partial x = I + \partial F / \partial x$.

The identity term means the gradient reaching x is at least as large as the gradient at y , no matter what F did. Stack ninety-six blocks and the gradient still has an unbroken path from the loss to the first layer.

This is the same additive trick as the LSTM's cell state in Volume VI — a path along which nothing multiplies.

READ THE BLOCK AS A STREAM, NOT A PIPELINE

A useful way to hold it: the residual path is a **shared workspace** that runs the full depth of the model. Each sub-layer reads the workspace, computes something, and *adds its contribution back*. Nothing ever overwrites what is already there.

This reframing — the “residual stream” — is now the standard lens in interpretability work: individual heads and FFN neurons are studied as things that write specific features into that stream, and later layers as things that read them.

JOB TWO — STOPPING THE REPRESENTATION FROM COLLAPSING

A pure attention stack drives every token's vector towards the same value. Dong, Cordonnier & Loukas (2021) showed the convergence is *doubly* exponential in depth — and that the residual path and the feed-forward network are precisely the two things that counteract it.

WORTH KNOWING — THE ORDERING MATTERS

$y = \text{LayerNorm}(x + F(x))$ puts a normalisation *on* the residual path. $y = x + F(\text{LayerNorm}(x))$ leaves the path clean. The second is what almost every model since GPT-2 uses, and the next two slides are about why.

FIGURE 21.2

What the residual is actually doing

Left: the gradient. Right: the representation.

Residual connections are not a training trick here — without them the model has nothing to say.

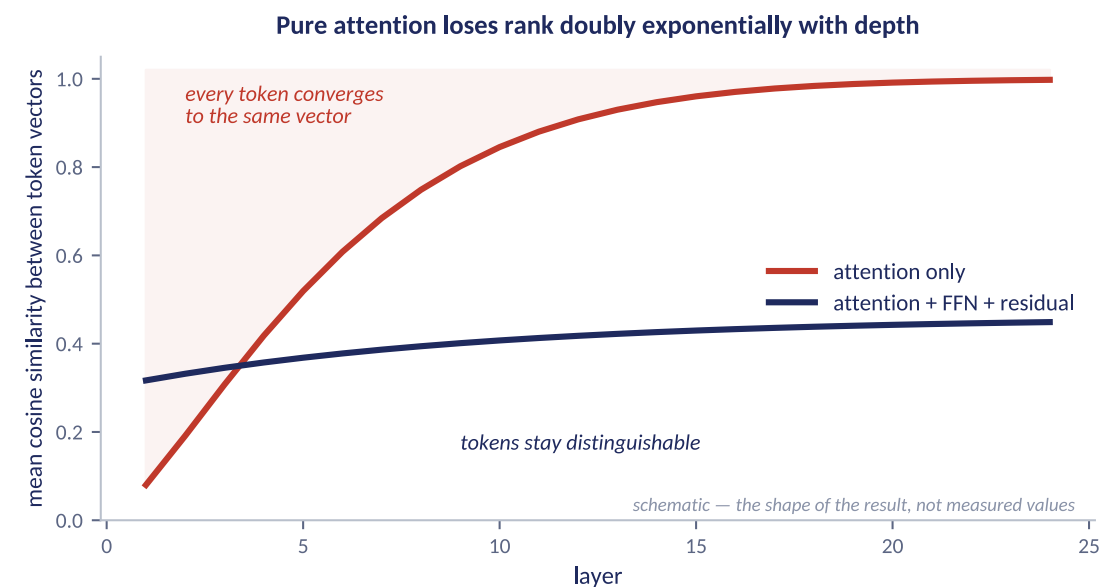
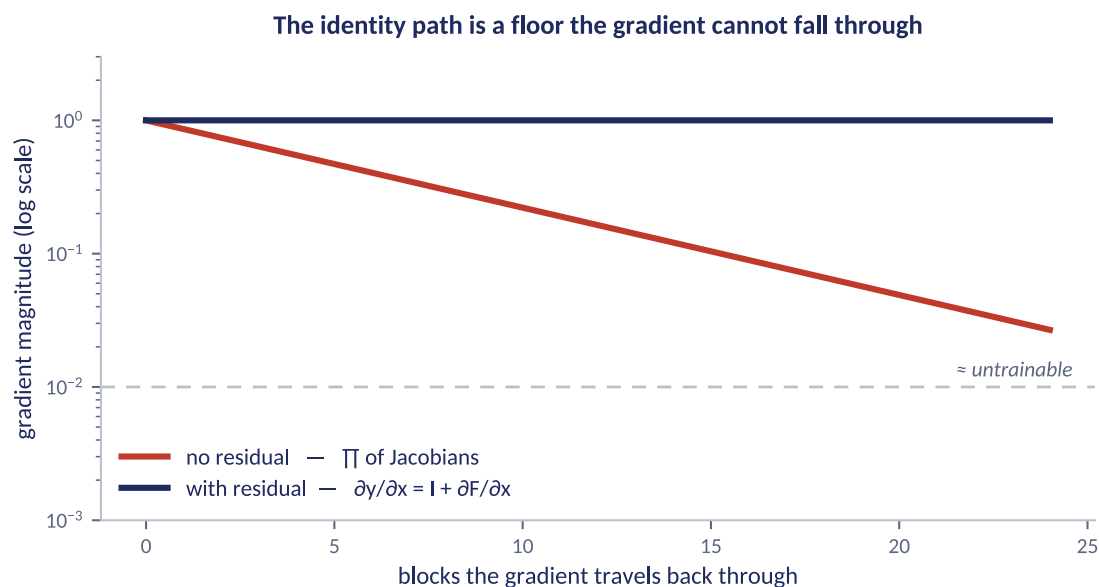


Figure 21.2: Two independent failure modes, one fix. The left panel is the familiar depth argument; the right is specific to attention, and is the reason “attention is all you need” is a claim about a whole block rather than a layer.

Normalise the vector, not the batch

Same idea as batch normalisation — subtract a mean, divide by a standard deviation, then re-scale with two learned parameters. The only difference is which numbers are averaged over, and that difference is everything.

FOR EACH POSITION INDEPENDENTLY

$$\mu = (1/d) \sum_i x_i \quad \sigma^2 = (1/d) \sum_i (x_i - \mu)^2$$

THEN NORMALISE AND RE-SCALE

$$y = \gamma \odot (x - \mu) / \sqrt{\sigma^2 + \epsilon} + \beta$$

γ and β are learned, one pair per dimension — $2d$ parameters, so the layer can undo the normalisation if that turns out to be what it wants.

WHY NOT BATCHNORM

- **Sequences have different lengths.** Batch statistics computed over padded positions are statistics of the padding.
- **Inference batches of one.** BatchNorm needs running averages that must match training; LayerNorm needs nothing but the vector in front of it.
- **Position-dependent distributions.** Token 1 and token 3000 have genuinely different activation statistics; a per-feature batch mean smears them together.

WORTH KNOWING — RMSNORM

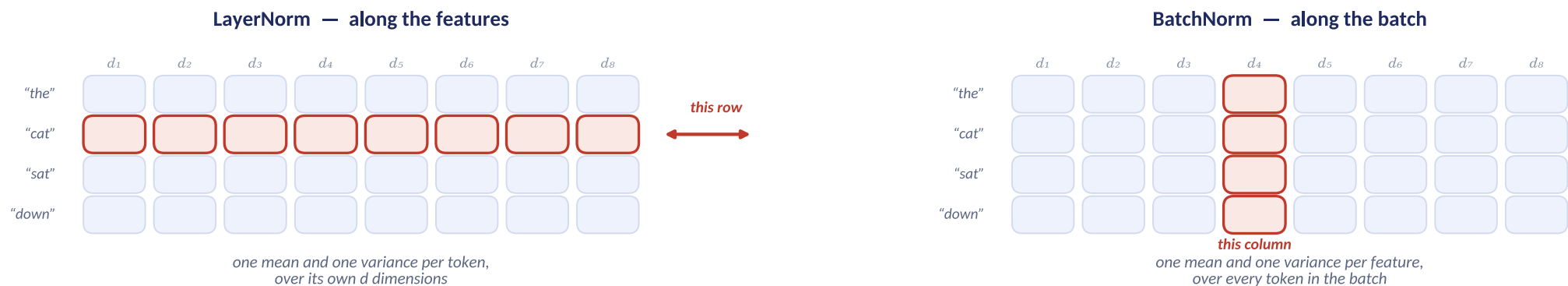
Most models since ~2020 (Llama, T5, PaLM) drop the mean subtraction and the bias entirely: $y = \gamma \odot x / \text{RMS}(x)$. It is measurably cheaper, and the re-centring turns out to contribute almost nothing. Same slot in the block, fewer operations.

FIGURE 21.3

Which axis gets normalised

The whole distinction, in two grids.

Same operation, different axis — and the axis is the whole reason one of them works here.



WHY BATCHNORM IS THE WRONG TOOL FOR A SEQUENCE

BatchNorm's statistics depend on the other examples in the batch and on how many real tokens each one has — and padded batches of variable-length sentences make those counts differ per position. LayerNorm depends on nothing but the vector in front of it: same result at batch size 1 or 1024, training or inference, position 3 or position 3000.

Figure 21.3: LayerNorm's statistics come from inside one token's own vector. That independence from the batch is what makes it usable at batch size 1, on variable-length input, at any position.

Post-LN, Pre-LN, and the warm-up mystery

The 2017 paper put the LayerNorm after the addition. Everything since has put it before the sub-layer. The change is one line, and it removed a notoriously fragile piece of the recipe.

Post-LN 2017

$$x = \text{LN}(x + \text{Sublayer}(x))$$

The residual path passes through a LayerNorm in every block, so the gradient is rescaled at each one on the way back. At depth this produces very large gradients in the top layers early in training.

Pre-LN GPT-2 onward

$$x = x + \text{Sublayer}(\text{LN}(x))$$

The residual path from input to output is now completely clean — no normalisation anywhere on it. Gradient magnitudes stay well behaved at any depth.

What it bought

Post-LN Transformers need a learning-rate **warm-up** to train at all; remove it and they diverge. Pre-LN trains without one. Post-LN, when it does converge, often ends slightly better — which is why the trade is occasionally still argued about.

THE GENERAL LESSON WORTH CARRYING

Warm-up was treated for years as a mysterious but necessary ritual. It was not: it was compensating for a specific architectural choice about where a normalisation sat. When a training recipe needs a step nobody can explain, that step is usually paying for a design decision somewhere else.

FIGURE 21.4

Where the LayerNorm sits

Two block diagrams, and the schedule one of them requires.

Where the LayerNorm sits decides whether the model needs a warm-up schedule.

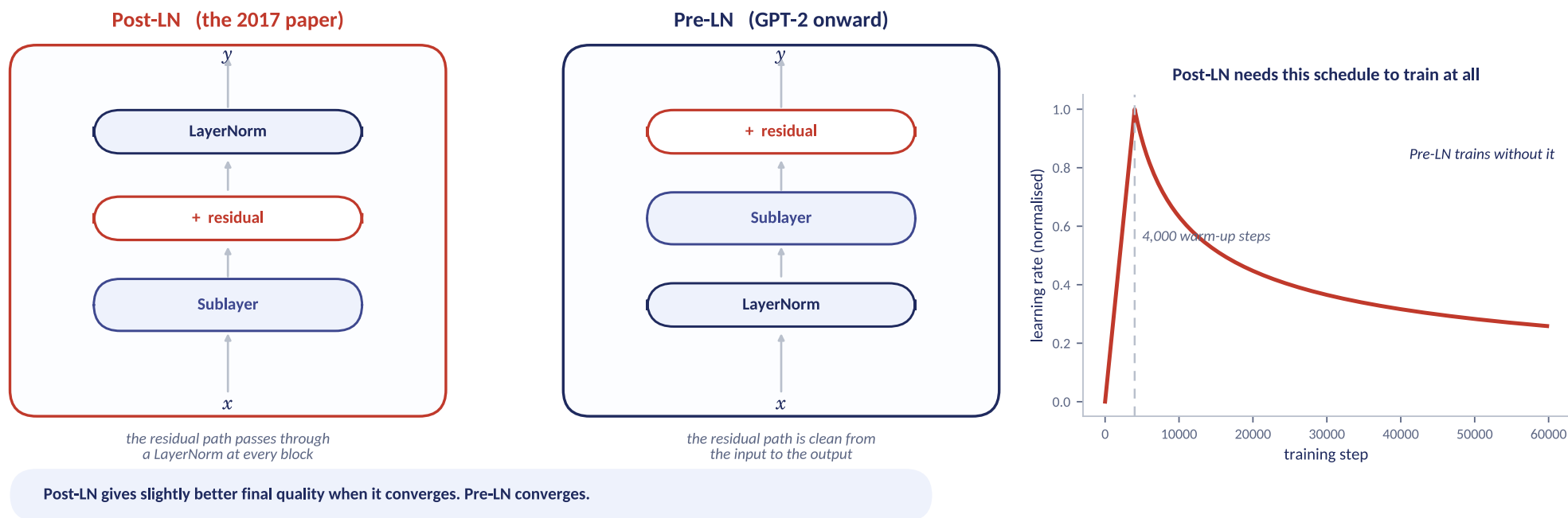


Figure 21.4: The inverse-square-root schedule with 4,000 warm-up steps is not a general truth about optimisation — it is what Post-LN needs. Pre-LN made it optional.

THE OTHER SUB-LAYER

**Attention mixes across positions.
Something else has to think about
them.**

Every operation in an attention layer is a weighted average or a projection. Nothing in it applies a non-linearity to a position on its own — and a model made only of weighted averages cannot represent very much.

A two-layer MLP, run at every position

The least glamorous component in the architecture, and the one that holds most of its parameters.

THE WHOLE THING

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x} \mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2$$

$\mathbf{W}_1$ is $d \times 4d$, $\mathbf{W}_2$ is $4d \times d$. Up to 2048, through a ReLU, back down to 512.

WHAT IT APPEARS TO BE DOING

A widely-supported reading is that the FFN acts as a **key-value memory**: $\mathbf{W}_1$'s rows behave like pattern detectors over the incoming vector, and $\mathbf{W}_2$'s columns like the content written back when a pattern fires. Much of the factual knowledge a language model has can be located in these layers, and targeted model-editing methods work by rewriting them.

THREE THINGS TO NOTICE

- **It is position-wise.** The same $\mathbf{W}_1$ and $\mathbf{W}_2$ are applied to every token separately. No information crosses between positions — that is attention's job, and only attention's.
- **It is where the non-linearity is.** Take the FFN out and the block is very nearly a sequence of linear maps.
- **It is four times wider in the middle.** The expansion is where the capacity lives; narrow it and quality falls off quickly.

WORTH KNOWING — THE MODERN VERSION

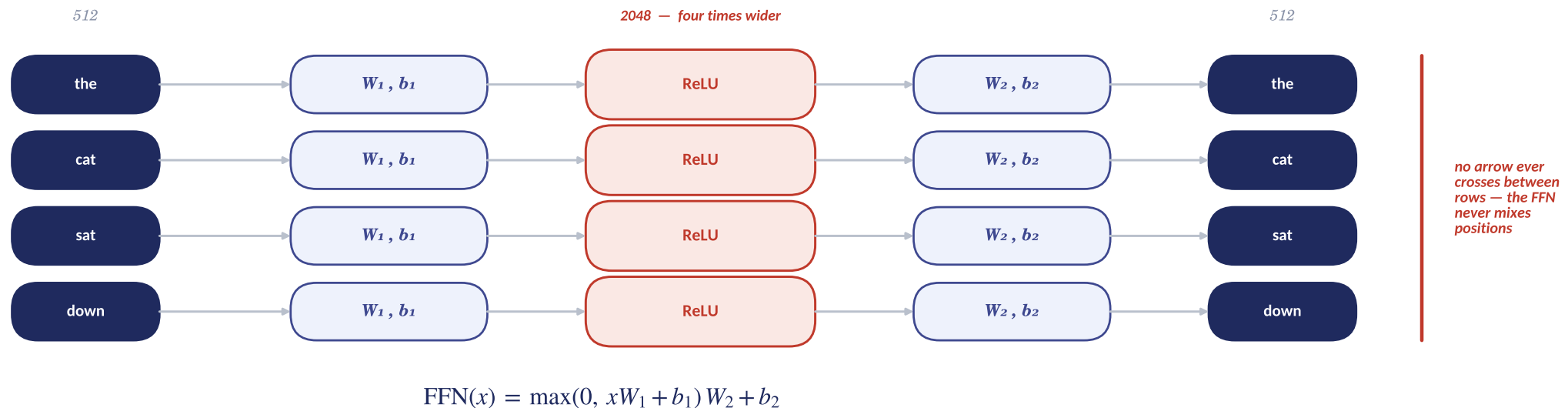
Almost every recent model replaces ReLU with a **gated** variant, usually SwiGLU: $(\mathbf{x}\mathbf{W} \cdot \sigma(\mathbf{x}\mathbf{V})) \mathbf{W}_2$, three matrices instead of two. The hidden width is cut to about $8d/3$ to keep the parameter count level. Modest, consistent quality gains — and the same slot in the same block.

FIGURE 21.5

Position-wise means exactly that

One tiny network, four independent applications.

The same tiny two-layer network, applied to every position separately.



THE DIVISION OF LABOUR

Attention is the only thing in the block that moves information between positions. The feed-forward network is the only thing that transforms a position on its own. One mixes, the other thinks — and roughly two-thirds of the block's parameters are in the half that does the thinking.

Figure 21.5: No arrow crosses between the rows. Attention is the only mechanism in the entire architecture that moves information between positions — everything else operates on one token at a time.

FIGURE 21.6

The encoder block, opened up

Everything from the last five slides, assembled.

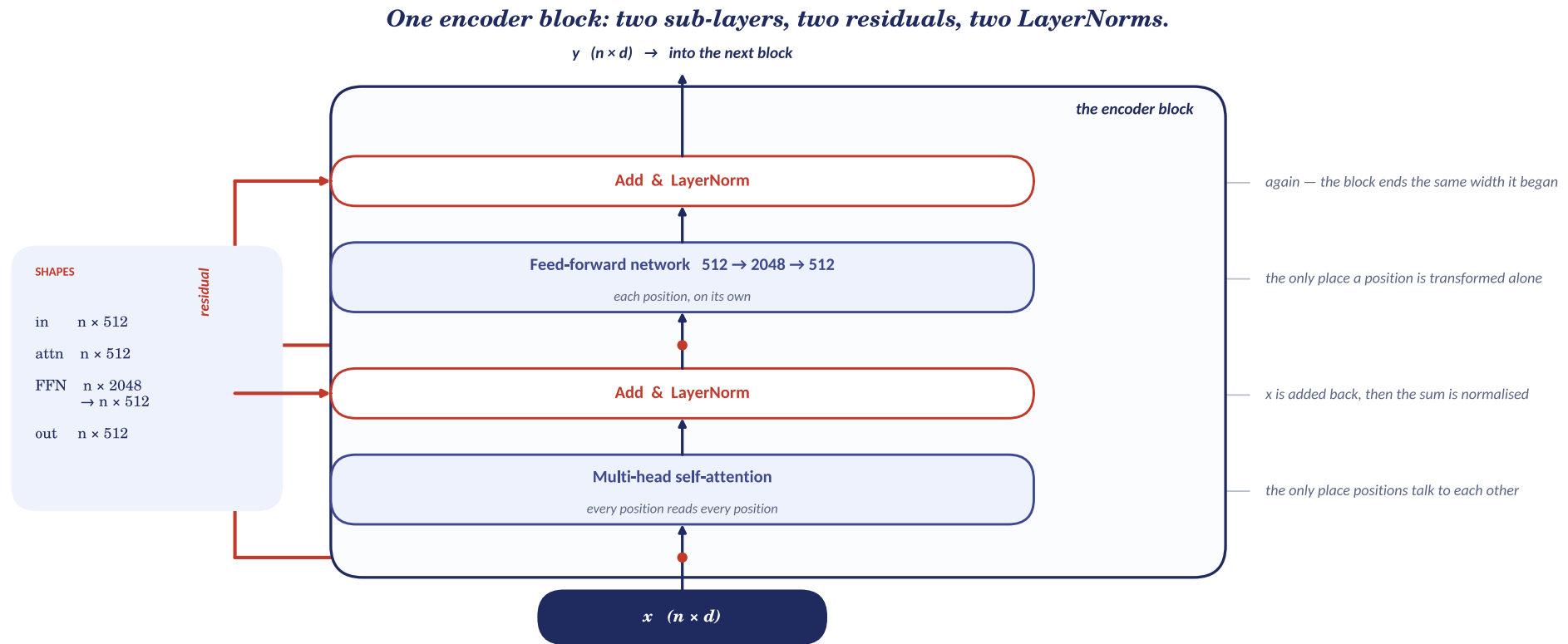


Figure 21.6: Two sub-layers, two residuals, two LayerNorms, one shape in and the same shape out. Six of these stacked is the encoder of the original paper; ninety-six of the decoder variant is a large language model.

Every parameter in one block

$d = 512$, $d_{\text{ff}} = 2048$, $h = 8$ heads. Nothing in this table depends on the sequence length.

Component	Shape	Parameters	What it contributes
W_Q, W_K, W_V	$3 \times (512 \times 512)$	786 k	The projections, summed across all eight heads — each head owns a 512×64 slice.
W_O	512×512	262 k	Mixes the concatenated heads back together.
W_1, b_1	512×2048	1.05 M	The expansion. Roughly a third of the block on its own.
W_2, b_2	2048×512	1.05 M	The projection back down to d .
$2 \times \text{LayerNorm}$	$2 \times (2 \times 512)$	2 k	γ and β . Negligible in count, not in effect.
One encoder block	—	3.15 M	One-third attention, two-thirds feed-forward. That ratio is fixed by the choice $d_{\text{ff}} = 4d$ and barely moves across model sizes.

Six encoder blocks plus six decoder blocks plus a 37,000-token embedding table give the 65 M parameters of Transformer-base. Change d to 4096 and stack thirty-two decoder blocks and the same table gives you roughly seven billion.

FIGURE 21.7

Where the parameters actually live

Attention gets the attention. The feed-forward network gets the parameters.

Attention is where the ideas are. The feed-forward network is where the memory is.

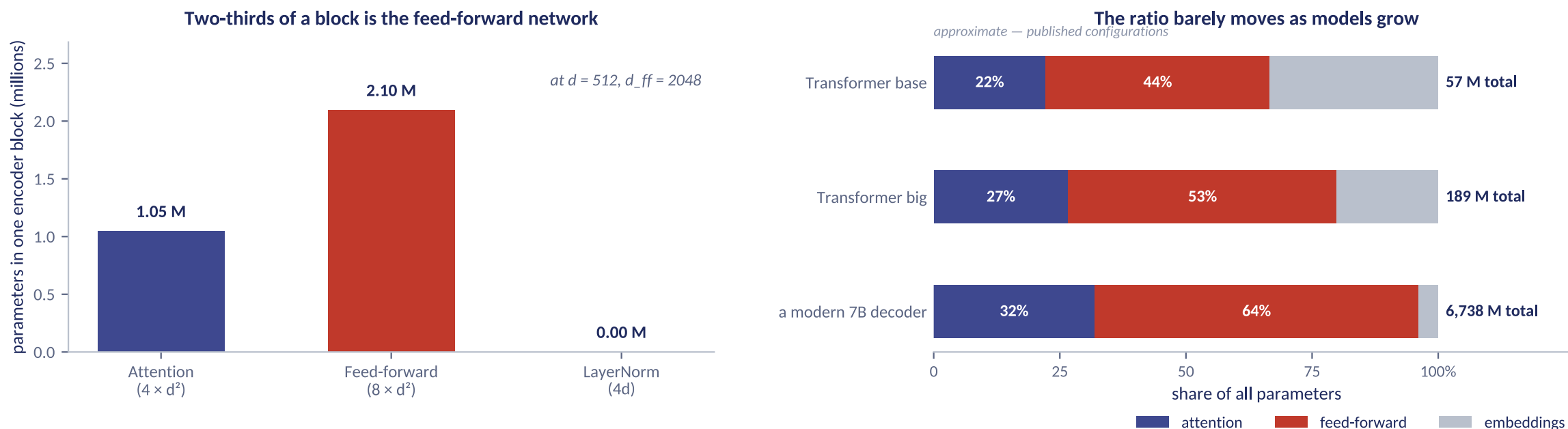


Figure 21.7: The split is a consequence of arithmetic, not of tuning: attention costs $4d^2$ per block and the feed-forward network $8d^2$. Any change to that ratio is a deliberate architectural decision.

What stacking the block actually buys

One block lets every position read every other position once. Two blocks let it read the *result* of everyone else's reading.

Composition of relations

Layer 1 can resolve "*it* → *animal*". Layer 2 can then use the enriched *it* vector to resolve something about the whole clause. Multi-hop reasoning over a sentence needs depth in the same way a proof needs steps.

A rough division of labour

Probing studies find a consistent gradient in trained models: surface form and morphology early, syntax in the middle, semantics and task-specific structure late. The architecture never asks for this — it falls out of training.

The residual stream grows

Because every sub-layer *adds* to the stream rather than replacing it, depth accumulates features rather than transforming them away. A late layer can still read something an early layer wrote.

THE CONFIGURATIONS WORTH REMEMBERING

MODEL	BLOCKS	D	HEADS	PARAMETERS
Transformer base	6 + 6	512	8	65 M
Transformer big	6 + 6	1024	16	213 M
BERT-base	12 enc	768	12	110 M
GPT-2 XL	48 dec	1600	25	1.5 B
a 7B decoder	32 dec	4096	32	7 B

DEPTH OR WIDTH?

Both, in a fixed relationship. Scaling studies find that models perform best when depth and width grow together — a very deep, very narrow Transformer and a very shallow, very wide one both underperform a balanced model of the same total size. Note also that every row above keeps $d/\text{heads} \approx 64$: the per-head width has barely changed in eight years.

THE SECOND STACK

The encoder reads. The decoder writes.

Everything so far describes a stack that turns a sequence into a better representation of itself. To produce a *different* sequence you need a second stack — one that can look at what it has written so far, and at what the first stack read.

The same block, with one sub-layer inserted

Three wrapped sub-layers instead of two. The first and third are already familiar; only the middle one is new.

1 Masked self-attention

Q, K, V from the decoder

Identical to the encoder's self-attention, plus the causal mask of Volume VII. It lets the decoder condition on everything it has generated so far, and forbids it from looking at the token it is being asked to predict.

2 Cross-attention

Q decoder, K and V encoder

The only place the two stacks touch. Queries come from the decoder's current state; keys and values come from the encoder's final output. This is Bahdanau attention from Volume VII, written in the Transformer's notation and given multiple heads.

3 Feed-forward

position-wise

The same two-layer network as the encoder, with its own weights. Applied to each position on its own.

and above the stack

once, not per block

A single linear projection from d to the vocabulary size, then a softmax — one distribution over the next token, at every position at once.

FIGURE 21.8

The decoder block, opened up

Where each sub-layer looks.

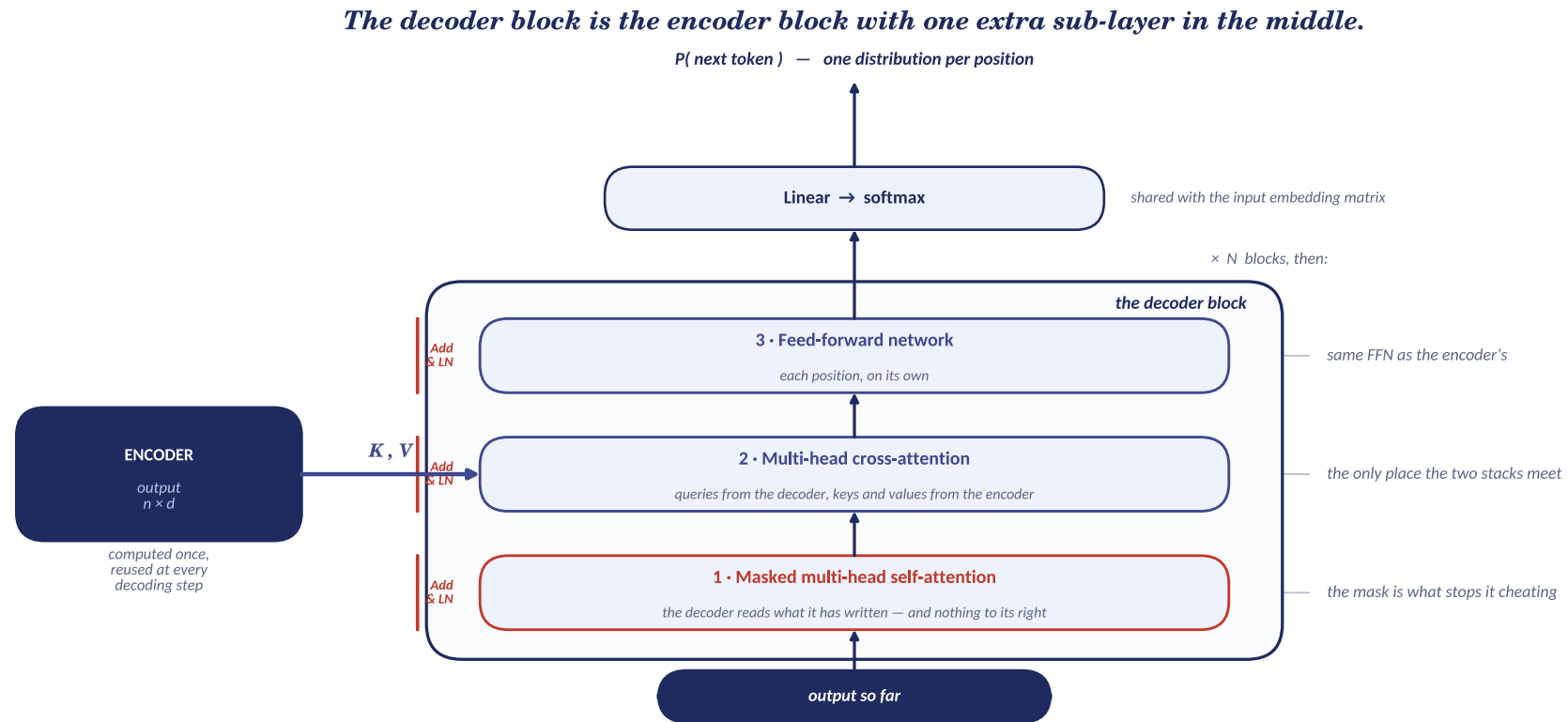


Figure 21.8: The encoder's output is computed once for the whole source sentence and fed to the cross-attention of every decoder block — it does not change while the decoder generates.

The same layer, with its inputs coming from two places

There is no third attention implementation. Encoder self-attention, decoder self-attention and cross-attention are one function called with different arguments.

WHAT THE QUERIES ARE ASKING

The decoder's state at position t encodes "here is what I have written, and here is what I am about to write." Its query goes out to the source sentence and asks which parts of it are relevant to that next word.

Plotting these weights recovers a soft word-alignment between source and target — the same picture Bahdanau produced in 2014, now free of any recurrence.

WHY K AND V ARE COMPUTED ONCE

The encoder's output does not depend on the decoder, so its keys and values are computed a single time and reused at every decoding step. In a translation system this is a large constant factor of saved work, and it is the reason encoder-decoder models remain competitive for translation and speech.

THE THREE ATTENTIONS, SIDE BY SIDE

WHERE	Q FROM	K, V FROM	MASK
Encoder self-attn	encoder	encoder	none
Decoder self-attn	decoder	decoder	causal
Cross-attention	decoder	encoder	none

NOTE WHAT IS NOT SYMMETRIC

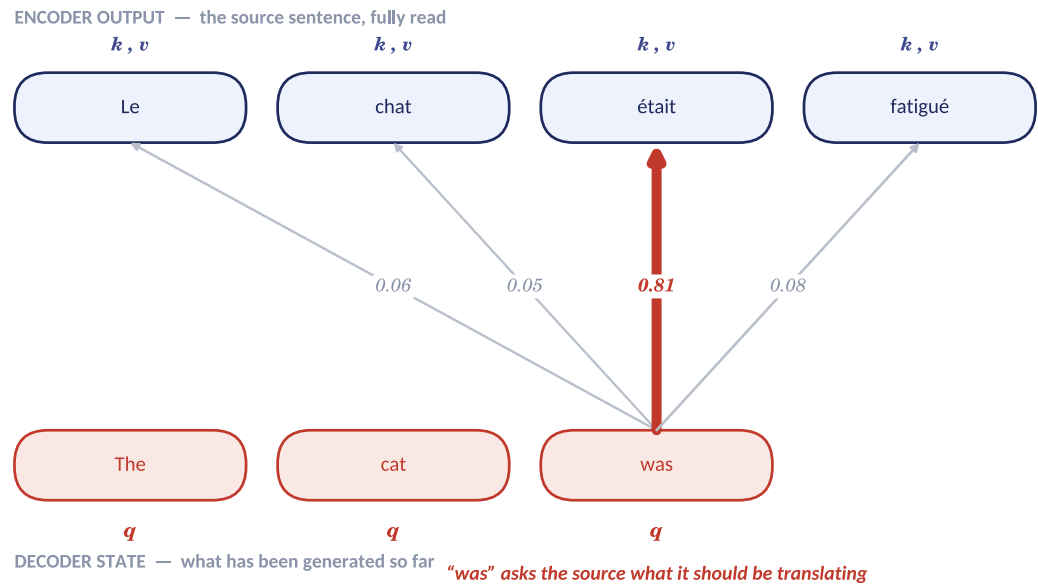
Cross-attention lets the decoder read the encoder. There is no path in the other direction — the encoder never sees the output being generated. That is what makes it safe to compute the encoder once, and it is also why the encoder's representation of the source is entirely task-agnostic.

FIGURE 21.9

Where Q, K and V come from

The only difference between the three attentions.

The same attention layer — the only change is where the three inputs come from.



THE ONLY DIFFERENCE BETWEEN THE THREE

- Encoder self-attention**
 $Q, K, V \leftarrow \text{encoder}$
no mask
- Decoder self-attention**
 $Q, K, V \leftarrow \text{decoder}$
causal mask
- Cross-attention**
 $Q \leftarrow \text{decoder}, K, V \leftarrow \text{encoder}$
no mask

Same equation, same code path. Only the arguments change.

Figure 21.9: The decoder position “was” queries the whole source sentence and finds “était”. Same scaled dot-product attention, same code — the queries simply came from a different stack.

FIGURE 21.10

The whole architecture

Everything in this chapter, on one page.

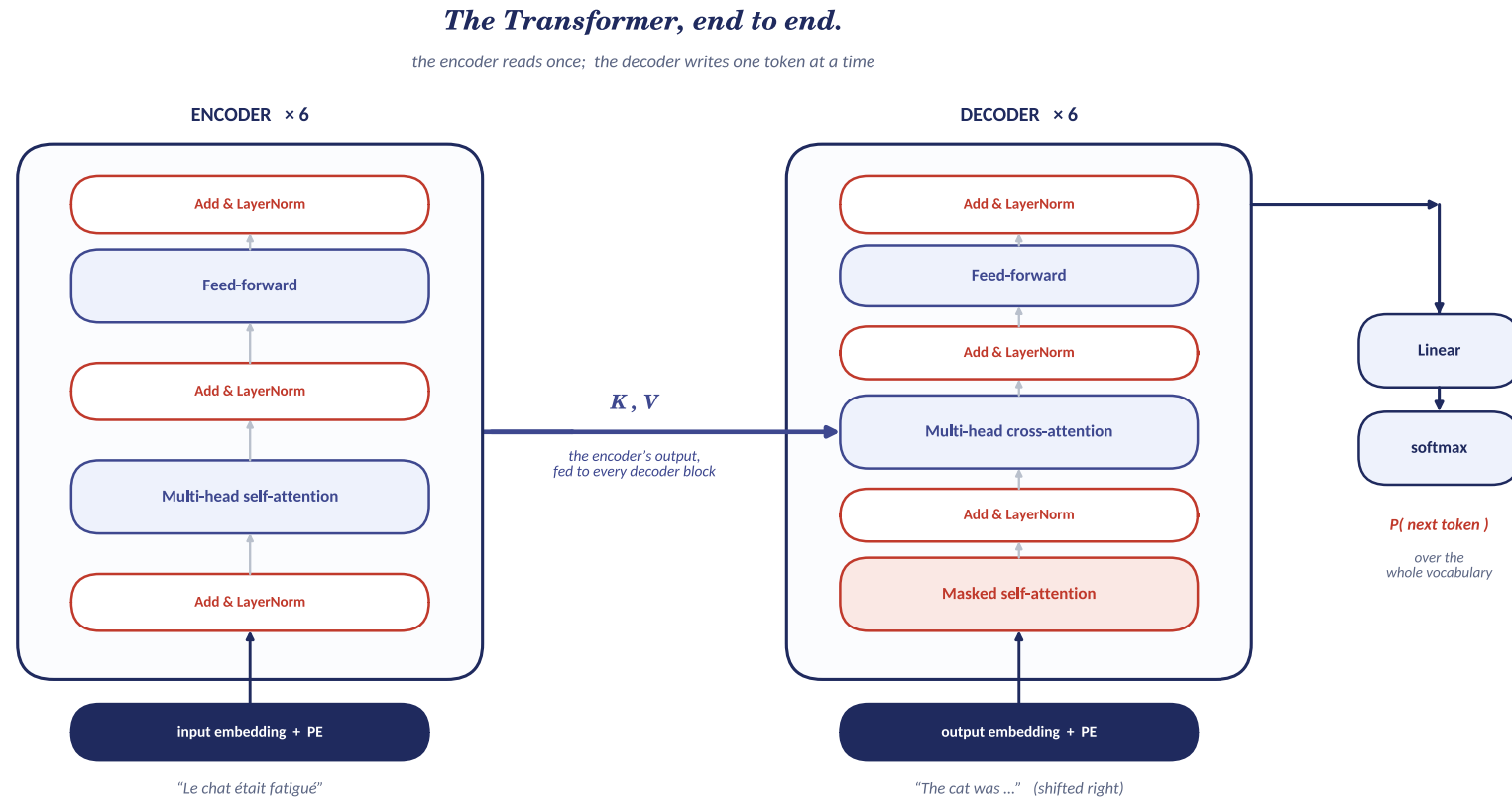


Figure 21.10: Two stacks of six blocks, one bridge between them, embeddings and positional encodings at the bottom, a linear projection and a softmax at the top. Nothing else is in the model.

Input and output, and one shared matrix

The stack is the interesting part, but three details at the boundaries are worth having straight.

Input embedding + position

Token ids become vectors through a lookup table, then the positional encoding of Volume VII is **added** — not concatenated. Position and meaning share the same 512 dimensions, and the model learns to keep them separable.

The $\sqrt{d_{\text{model}}}$ scaling

Embeddings are multiplied by $\sqrt{d}$ before the positional encoding is added. The embedding matrix is initialised with variance $\approx 1/d$, so this puts it on the same scale as the sinusoids, which are $O(1)$. Without it the position signal drowns the token.

Weight tying

The input embedding, the decoder's output embedding and the final linear layer share one matrix. It saves a large number of parameters, and it makes intuitive sense: mapping a word to a vector and scoring a vector against a word are the same relationship, read in two directions.

ONE CONSEQUENCE WORTH NOTICING

Because of weight tying, the vocabulary appears in the parameter count exactly once. For Transformer-base that embedding table is around 19 M of 65 M parameters — nearly a third of the model is the vocabulary. At 7 B parameters the same table is under 4%, which is one of several reasons large models can afford much larger vocabularies.

Trained in parallel, generated in sequence

The property that made the Transformer trainable at scale does not survive to inference — and that asymmetry shapes everything about how these models are served.

TRAINING — TEACHER FORCING

The correct output sequence is already known, so it is fed to the decoder in full, shifted right by one. The causal mask makes every position's prediction depend only on the true tokens to its left. One forward pass produces n predictions and n losses.

An RNN gets exactly the same n training signals — but needs n sequential steps to collect them.

THE KV CACHE

Every step re-derives identical keys and values for every token already in the context. Caching them turns each new token from $O(n^2)$ work into $O(n)$.

The cost is memory: one cache per layer, per head, growing with the context. At long context lengths that cache — not the weights — is what fills the GPU, which is why grouped-query attention (share one set of K and V across several query heads) is now standard.

INFERENCE — AUTOREGRESSIVE DECODING

The output does not exist yet. Generate one token, append it, run the model again. n tokens means n forward passes, and the sequential dependency the architecture removed from training is back.

EXPOSURE BIAS — THE HONEST CAVEAT

During training the decoder only ever conditions on *correct* prefixes. At generation it conditions on its own output, mistakes included, and those mistakes compound. Beam search, sampling temperature and the whole decoding literature exist largely to manage this gap.

FIGURE 21.11

One pass, then n passes

Training and generation, drawn to the same scale.

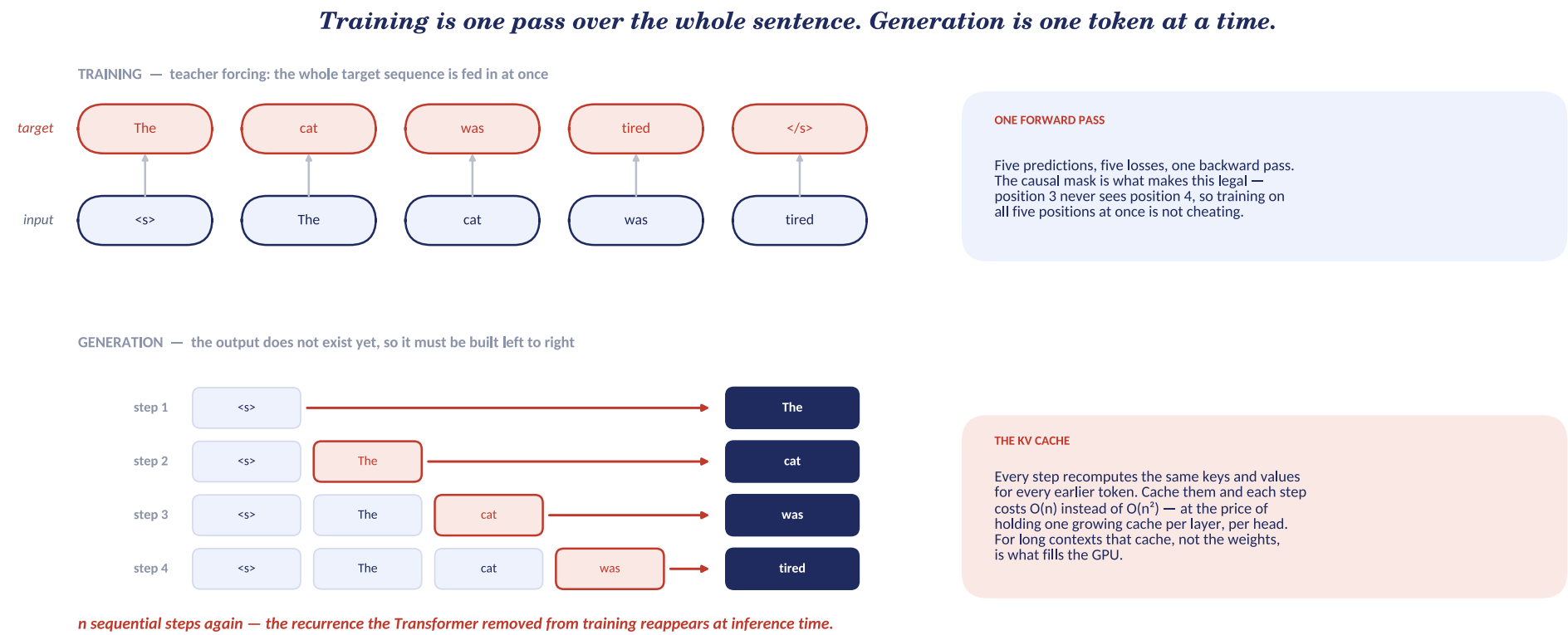


Figure 21.11: Parallel training is the property that made pre-training on internet-scale corpora feasible. Sequential generation is the property that makes serving these models expensive.

The 2017 configuration, and the result

Reproduced because the numbers are small enough to be surprising — this was not an expensive experiment by the standards of anything that followed.

The recipe	Value
Optimiser	Adam, $\beta_2 = 0.98$
Learning rate	inverse-square-root, 4,000 warm-up steps
Dropout	0.1, on every sub-layer output and on the embeddings
Label smoothing	0.1 — hurts perplexity, helps BLEU
Batching	by token count, ~25,000 tokens per batch
Decoding	beam search, width 4, length penalty 0.6

The result, WMT 2014 EN→DE	BLEU	Training cost
Best previous single model	25.2	~10× more
Best previous ensemble	26.3	~10× more
Transformer base	27.3	12 hours, 8 GPUs
Transformer big	28.4	3.5 days, 8 GPUs

THE NUMBER THAT MATTERED

Not the BLEU score — a point of BLEU is a good result, not a revolution. The right-hand column is the finding: state of the art at roughly a tenth of the training cost, on hardware anyone had. Everything that followed was downstream of that ratio.

ONE BLOCK, THREE FAMILIES

Almost nobody uses the architecture in the paper.

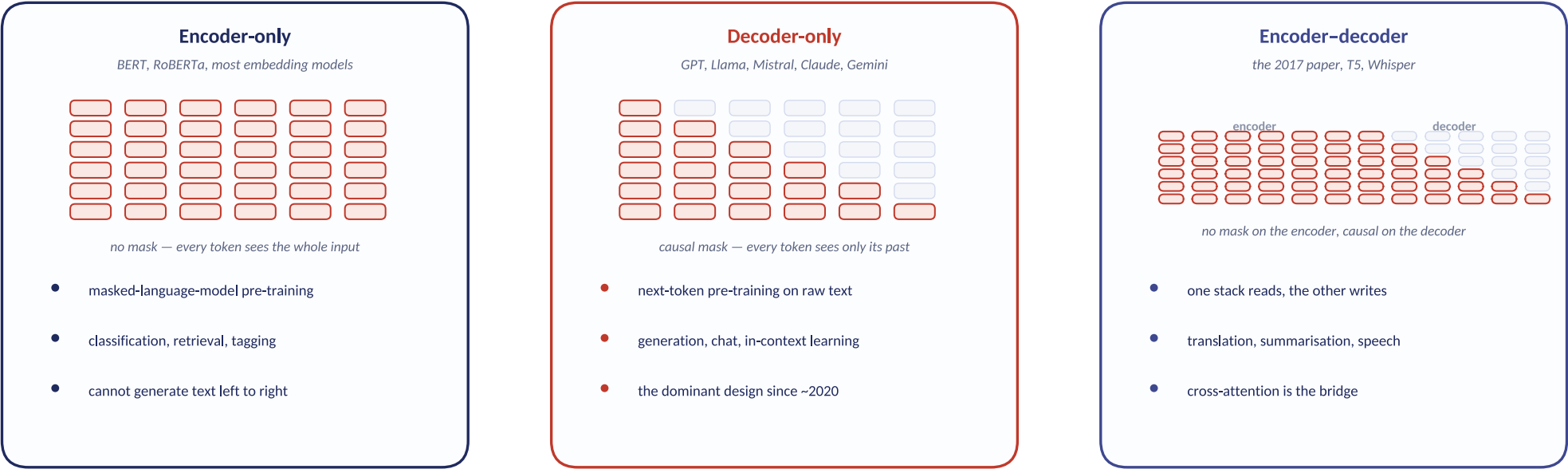
Within three years the field had split the encoder-decoder in half and mostly kept one side. The block never changed — what changed was how many stacks there were and which entries of the score matrix were allowed to be non-zero.

FIGURE 21.12

The three families

Told apart by their mask.

One block. Three ways to wire it — and the mask is what names them.



The block never changes. What changes is how many stacks there are, and which entries of the score matrix are allowed to be non-zero.

Figure 21.12: Encoder-only sees everything and generates nothing; decoder-only sees only the past and generates fluently; encoder-decoder does both with two stacks and a bridge.

Why decoder-only won

Not because it is architecturally superior. Because of what it can be trained on.

Family	Pre-training objective	Where it wins, and why
Encoder-only BERT, RoBERTa	Mask 15% of the tokens, predict them from both directions.	Bidirectional context makes it the stronger <i>reader</i> : classification, retrieval, token tagging, and almost every production embedding model. It cannot generate text.
Decoder-only GPT, Llama, Claude	Predict the next token. Every token in the corpus is a training example, with no masking scheme to design.	The objective is the cheapest and densest available — which is what let it scale. Generation, chat, and in-context learning came out of scale rather than out of the architecture.
Encoder-decoder T5, Whisper, NMT	Span corruption, or supervised sequence-to-sequence pairs.	Best when input and output are genuinely different objects and the input is read once: translation, speech recognition, structured transduction. The separation lets the encoder be bidirectional while the decoder stays causal.

THE UNDERRATED REASON

A decoder-only model needs no distinction between input and output — a prompt and its continuation are the same sequence. That collapses every task into one format, which is what makes instruction tuning and in-context learning possible at all. The architecture did not become general; the *interface* did.

The block, eight years on

The skeleton is untouched. Nearly every component inside it has been replaced by something slightly better.

INSIDE THE BLOCK

- **Pre-LN** instead of Post-LN — no warm-up needed.
- **RMSNorm** instead of LayerNorm — no mean subtraction, fewer operations.
- **SwiGLU** instead of ReLU in the feed-forward network — gated, three matrices.
- **RoPE** instead of sinusoidal positional encoding — rotate Q and K by a position-dependent angle, so the score depends on relative offset.
- **No biases** in most linear layers — they contribute little and cost bandwidth.

AROUND THE ATTENTION

- **Grouped-query attention** — several query heads share one set of K and V, shrinking the KV cache several-fold.
- **FlashAttention** — mathematically identical, but never materialises the $n \times n$ matrix; an IO-aware kernel rather than an approximation.
- **Sparse and sliding-window patterns** — restrict which pairs are scored at all.
- **Mixture-of-experts** — many feed-forward networks per block, two of them active per token: parameter count and compute decoupled.

THE PATTERN IN THAT LIST

Every entry is a local substitution into a slot the 2017 block already defined. None of them changes the shape of the block, the residual stream, or the alternation of “mix across positions, then think about each position”. Eight years of intense, well-funded effort has improved the components and left the structure alone — which is a stronger statement about the design than any benchmark.

Key takeaways

- 1 A Transformer is one block, repeated. Two wrapped sub-layers in the encoder, three in the decoder, and nothing else — the whole architecture is that block plus embeddings and an output head.
- 2 Every sub-layer is written $y = \text{LayerNorm}(x + \text{Sublayer}(x))$. The residual is a protected gradient path *and* the thing that stops the representation collapsing; the LayerNorm normalises each token's own vector, which is why batch statistics never enter.
- 3 Attention is the only component that moves information between positions. The feed-forward network is the only one that transforms a position on its own — and it holds about two-thirds of the parameters.
- 4 The decoder adds cross-attention: queries from the decoder, keys and values from the encoder's output, computed once and reused. It is Bahdanau attention in Transformer notation.
- 5 Training is one parallel pass with teacher forcing and a causal mask; generation is n sequential passes with a KV cache. The parallelism that made pre-training possible does not survive to inference.
- 6 The mask is what names the family: no mask is BERT, causal is GPT, both is the original paper. Decoder-only won because next-token prediction is the cheapest objective and the most general interface — not because the architecture is better.

End of Volume VIII

Companion to Part VIII of the ML / DL Handbook

SOURCES

Vaswani et al., “Attention Is All You Need”, NeurIPS 2017 · arXiv:1706.03762

Ba, Kiros & Hinton, “Layer Normalization”, 2016 · arXiv:1607.06450

Xiong et al., “On Layer Normalization in the Transformer Architecture”, ICML 2020 · arXiv:2002.04745

Dong, Cordonnier & Loukas, “Attention Is Not All You Need”, ICML 2021 · arXiv:2103.03404

Geva et al., “Transformer Feed-Forward Layers Are Key-Value Memories”, EMNLP 2021 · arXiv:2012.14913

Shazeer, “GLU Variants Improve Transformer”, 2020 · arXiv:2002.05202 · Su et al., “RoFormer / RoPE”, 2021 · arXiv:2104.09864

Dao et al., “FlashAttention”, NeurIPS 2022 · arXiv:2205.14135